

Аутентификация во frontend-приложении

Auth state, protected pages, route guards, 401/403 и восстановление сессии

Результаты занятия

- ✓ Различать auth state интерфейса и серверную Authentication
- ✓ Моделировать состояния guest, loading и authenticated
- ✓ Создавать централизованный authState
- ✓ Реализовывать protected pages и route guards
- ✓ Создавать role-aware navigation
- ✓ Правильно обрабатывать 401 и 403
- ✓ Реализовывать logout UX
- ✓ Понимать проблему reload при in-memory token
- ✓ Понимать назначение auth bootstrap
- ✓ Сравнивать in-memory JWT и HttpOnly cookie/BFF архитектуру

Учебные вопросы

- 1 Auth state frontend** guest, loading, authenticated и единый источник состояния
- 2 Protected pages** Route guards и role-aware интерфейс
- 3 Жизненный цикл сессии** Login, protected request, 401, 403 и logout
- 4 Reload и auth bootstrap** Почему память теряется и как восстанавливают session
- 5 Безопасная browser architecture** In-memory token, HttpOnly cookie и BFF



Учебный вопрос 1. Auth state frontend

Frontend должен явно понимать, находится пользователь в системе или нет

Что такое auth state

- Auth state — состояние frontend, описывающее текущий статус аутентификации и данные пользователя, необходимые интерфейсу.
- Auth state нужен для отображения правильного UI.
- Он не заменяет server-side Authentication.
- Frontend может ошибиться или быть изменён пользователем.
- Backend остаётся источником истины для доступа к данным.

Три базовых состояния

● guest

Пользователь не аутентифицирован; показываем login/register.

● loading

Frontend ещё определяет состояние пользователя или ждёт API.

● authenticated

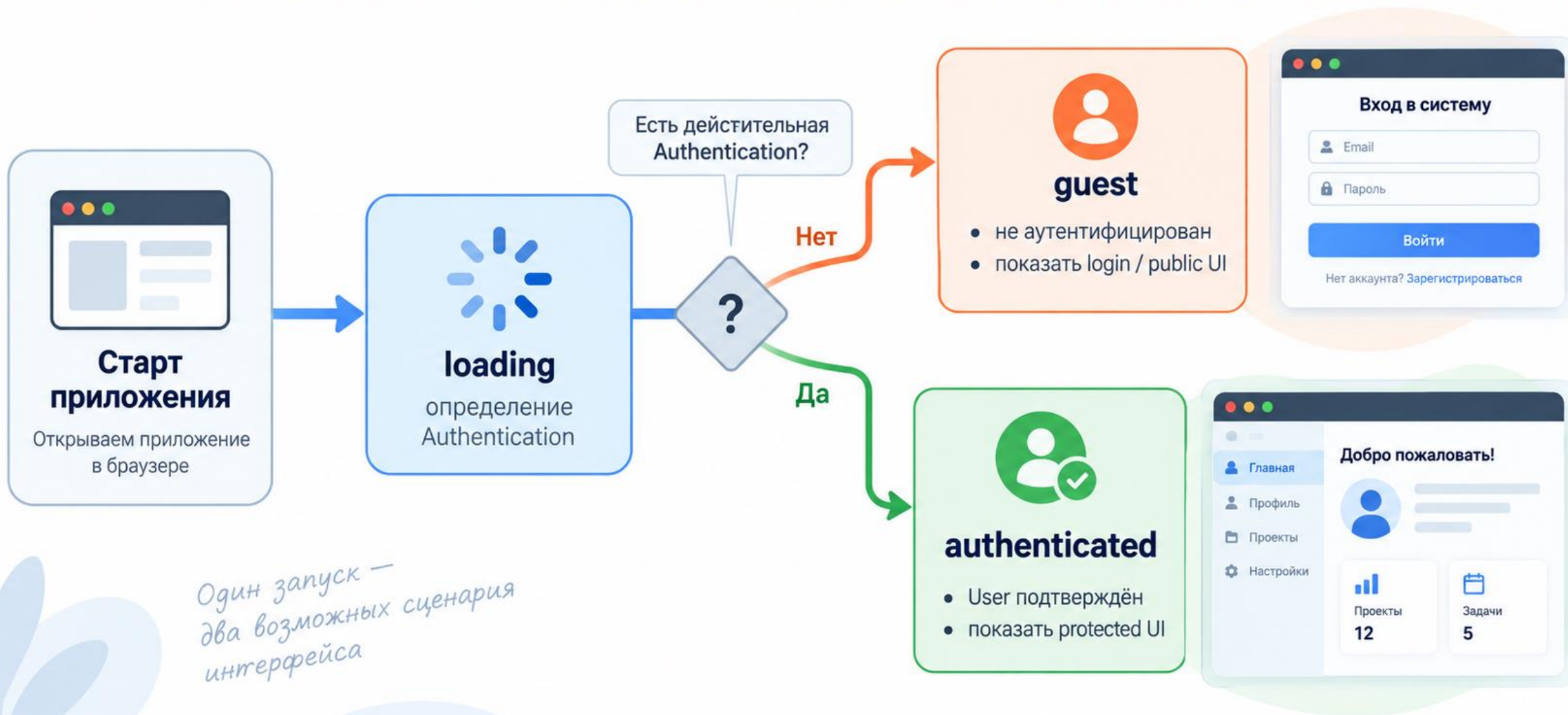
Есть подтверждённый User; показываем protected UI.

Зачем нужно loading

- Без loading интерфейс может сначала показать guest UI, а затем резко переключиться.
- При bootstrap frontend может ожидать ответ ``/api/auth/me``.
- Loading предотвращает ложные переходы и визуальное мерцание.
- В in-memory JWT варианте loading обычно короткий, но модель всё равно полезна.

Auth state machine: loading → guest или authenticated

При запуске приложение определяет состояние аутентификации и показывает соответствующий интерфейс



authState.js

JAVASCRIPT

```
const state = {
  status: "guest",
  user: null,
  accessToken: null
};

export function getAuthState() {
  return state;
}

export function setAuthenticated(user, accessToken) {
  state.status = "authenticated";
  state.user = user;
  state.accessToken = accessToken;
}

export function setGuest() {
  state.status = "guest";
  state.user = null;
  state.accessToken = null;
}
```

Что важно

- status явно описывает состояние.
- user содержит данные для UI.
- accessToken хранится только в памяти текущей страницы.
- Все компоненты используют один auth state.

Single source of truth

- Не создавайте отдельные переменные `isLoggedIn` в разных компонентах.
- Иначе части интерфейса могут показывать противоречивое состояние.
- Централизованный `authState` упрощает `login`, `logout` и обработку 401.
- Во фреймворках ту же роль выполняют `store`, `context` или `state manager`.



Учебный вопрос 2. Protected pages и route guards

Route guard управляет навигацией, но не заменяет backend authorization

Protected page

- Protected page — страница или представление, которое frontend показывает только пользователю в подходящем auth state.
- Пример: Мои заказы.
- Пример: Профиль.
- Пример: Admin panel.
- UI проверяет state перед отображением.
- API всё равно повторно проверяет JWT и права.

Страницы Internet Shop

Route	Guest	User	Admin
#/	Доступ	Доступ	Доступ
#/login	Доступ	Redirect	Redirect
#/orders	Redirect login	Доступ	Доступ
#/profile	Redirect login	Доступ	Доступ
#/admin	Redirect login	Forbidden UI	Доступ

requireUser()

JAVASCRIPT

```
export function requireUser() {  
  const auth = getAuthState();  
  
  if (auth.status !== "authenticated") {  
    location.hash = "#/login";  
    return false;  
  }  
  
  return true;  
}
```

Что важно

- Guard проверяет frontend auth state.
- Guest перенаправляется на login.
- Guard улучшает UX, но не защищает API.

requireAdmin()

JAVASCRIPT

```
export function requireAdmin() {
  const auth = getAuthState();

  if (auth.status !== "authenticated") {
    location.hash = "#/login";
    return false;
  }

  if (auth.user.role !== "admin") {
    renderForbidden();
    return false;
  }

  return true;
}
```

Что важно

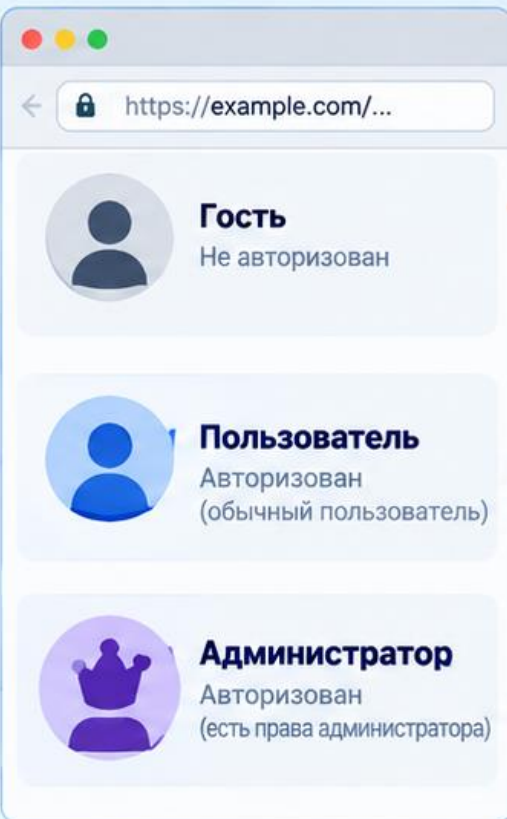
- Сначала проверяется Authentication.
- Затем frontend проверяет role для отображения UI.
- Backend всё равно обязан использовать requireRole(admin).

Protected page guard: guest → login, user → page, admin → admin page

Как работает защита страниц на frontend (например, в React / Vue / Angular)

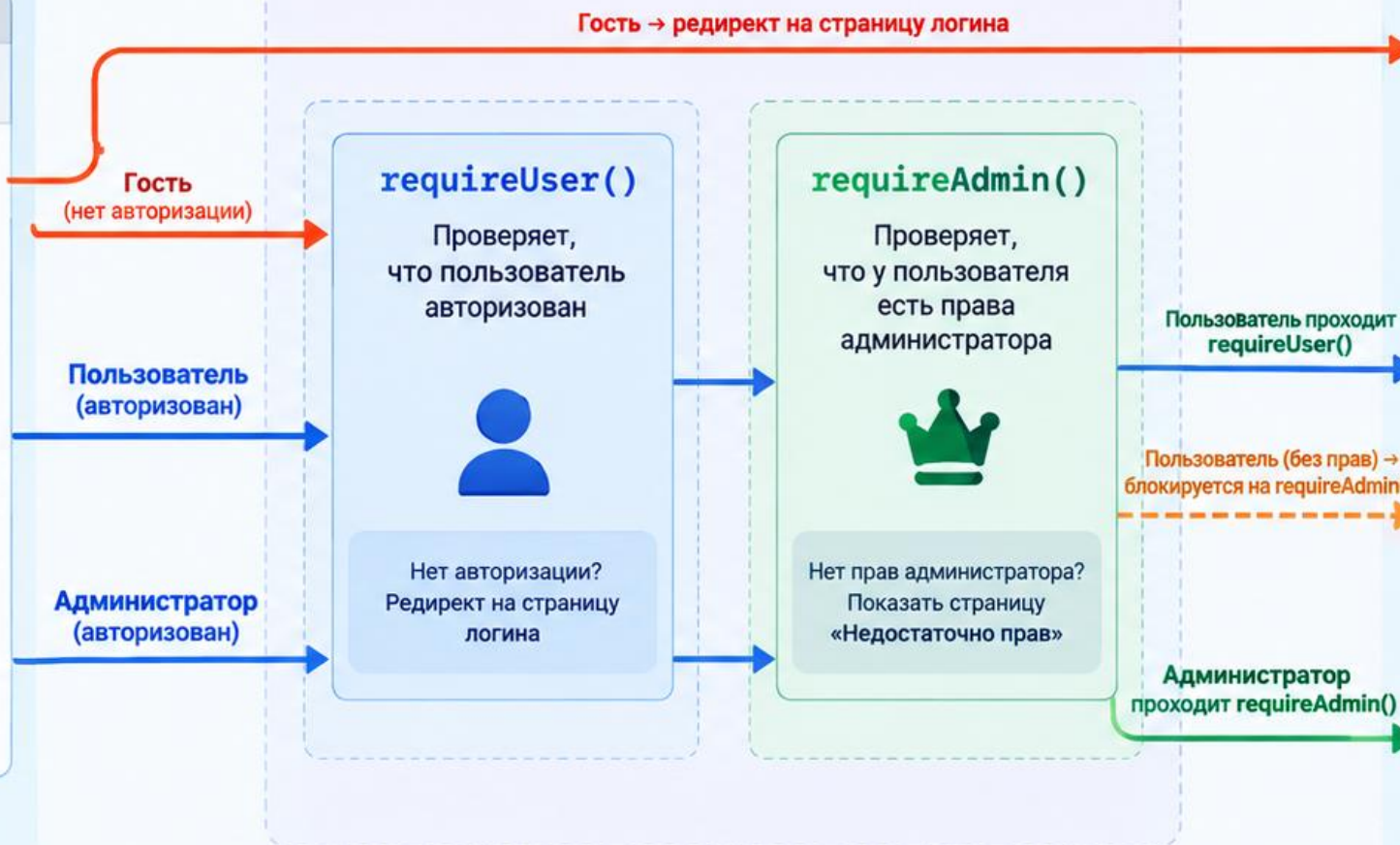
Пользователь в браузере

Открывает защищённую страницу



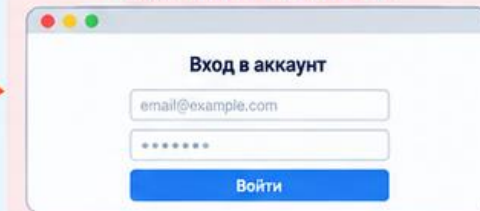
Frontend (защита маршрутов)

Проверяем состояние пользователя на клиенте

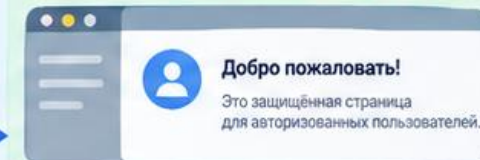


Страницы (результат на frontend)

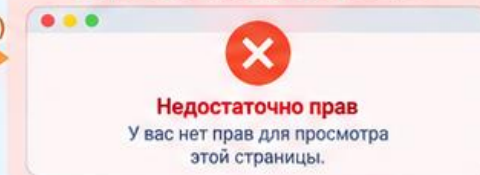
Страница входа (Login)



Обычная защищённая страница



Страница «Доступ запрещён»



Админ-панель



Frontend guard улучшает UX,
но backend всё равно проверяет доступ!

- ✓ Frontend guard позволяет быстрее реагировать и показывать понятные сообщения.
- ✓ Но настоящая защита выполняется на backend.
- ✓ Нельзя полагаться только на frontend — пользователь может обойти его (например, через API).



Настоящая защита
выполняется
на backend

Почему route guard не является защитой

- Пользователь может изменить frontend JavaScript.
- Можно вручную вызвать REST API через DevTools, curl или Postman.
- Можно изменить location.hash без использования меню.
- Настоящая защита реализуется requireAuth, requireRole и ownership checks на backend.
- Frontend guard отвечает за навигацию и UX.

Role-aware navigation

- Guest видит Login и Register.
- Authenticated User видит Profile, My Orders и Logout.
- Admin дополнительно видит Admin Panel.
- Скрытие пункта меню не выдаёт права — оно только отражает серверную модель доступа.

renderNavigation()

JAVASCRIPT

```
function renderNavigation() {  
  const auth = getAuthState();  
  
  loginLink.hidden = auth.status === "authenticated";  
  logoutButton.hidden = auth.status !== "authenticated";  
  ordersLink.hidden = auth.status !== "authenticated";  
  profileLink.hidden = auth.status !== "authenticated";  
  
  adminLink.hidden = !(  
    auth.status === "authenticated" &&  
    auth.user.role === "admin"  
  );  
}
```

Что важно

- Навигация строится из единого auth state.
- Admin link показывается только для role admin.
- Backend всё равно независимо проверяет права.



Учебный вопрос 3. Жизненный цикл сессии

Frontend должен согласованно реагировать на login, logout и ответы API

Login lifecycle

- 1 Открыть login** Guest видит форму входа.
- 2 Отправить credentials** Frontend вызывает POST /api/auth/login.
- 3 Получить User и token** Backend подтверждает Authentication.
- 4 Обновить authState** status становится authenticated.
- 5 Перерисовать UI** Появляются protected navigation и страницы.

login()

JAVASCRIPT

```
export async function login(email, password) {  
  const data = await apiFetch("/auth/login", {  
    method: "POST",  
    body: { email, password }  
  });  
  
  setAuthenticated(data.user, data.accessToken);  
  renderNavigation();  
  location.hash = "#/orders";  
}
```

Что важно

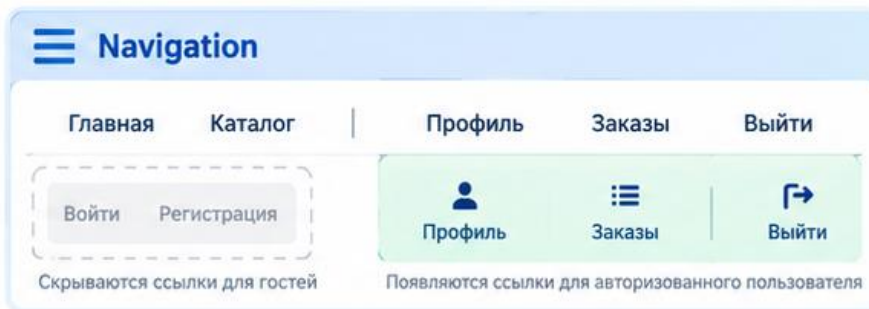
- После успешного login обновляется единый state.
- Навигация сразу отражает новое состояние.
- Пользователь переходит на protected page.

Protected request lifecycle

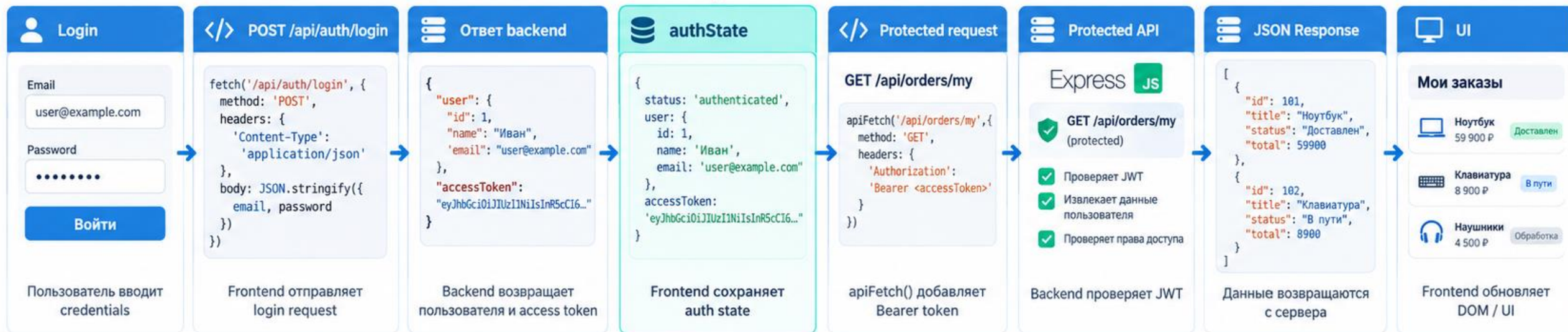
- Компонент вызывает `apiFetch()`.
- `apiFetch` берёт token из `authState`.
- Authorization: Bearer добавляется автоматически.
- Backend проверяет JWT.
- Успех возвращает JSON, а ошибка меняет поведение UI.

Frontend auth lifecycle: login → state → protected request → response

От входа пользователя до обновления интерфейса



Интерфейс меняется после login



Ключевая идея:

1

Login создаёт authenticated state

2

authState управляет navigation и protected requests

3

Backend остаётся источником истины

401 и 403 во frontend

401 Unauthorized

- Authentication недействительна
- Token отсутствует/expired/invalid
- Очистить auth state
- Перейти к login
- Можно показать: Сессия истекла

403 Forbidden

- User остаётся authenticated
- Недостаточно прав
- Не очищать auth state
- Показать сообщение или forbidden page
- Оставить доступ к разрешённым разделам

Централизованная реакция на 401

JAVASCRIPT

```
if (response.status === 401) {  
  setGuest();  
  renderNavigation();  
  location.hash = "#/login";  
  
  throw new APIError(  
    "Сессия истекла. Войдите снова.",  
    401  
  );  
}
```

Что важно

- Authentication state очищается централизованно.
- UI возвращается в guest.
- Пользователь получает понятное сообщение.

Реакция на 403

JAVASCRIPT

```
if (error.status === 403) {  
  renderMessage("Недостаточно прав");  
  return;  
}
```

Что важно

- User остаётся authenticated.
- Access token не удаляется.
- Недоступно только конкретное действие или страница.

Logout UX

- Logout должен немедленно изменить интерфейс.
- Очистить User и token из frontend state.
- Удалить sensitive data из DOM.
- Перейти на публичную страницу.
- При server-managed session дополнительно вызывается backend logout endpoint.

logout() для in-memory JWT

JAVASCRIPT

```
export function logout() {  
  setGuest();  
  clearProtectedContent();  
  renderNavigation();  
  location.hash = "#/";  
}
```

Что важно

- В учебной схеме удаляется локальный accessToken.
- После logout protected UI исчезает.
- JWT-копия, если была украдена, может оставаться валидной до exp.



Учебный вопрос 4. Reload и auth bootstrap

Перезагрузка страницы уничтожает in-memory state

Что происходит при F5

- JavaScript-контекст создаётся заново.
- Переменные `authState` возвращаются к начальным значениям.
- In-memory `accessToken` исчезает.
- Frontend больше не может отправить Bearer token.
- Пользователь визуально становится guest.

Reload: in-memory JWT исчезает → frontend снова guest

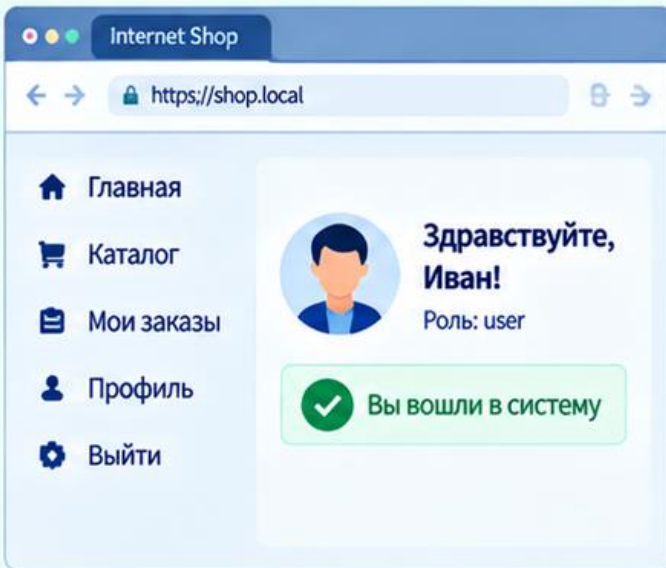
Токен хранится только в памяти JavaScript. После перезагрузки страницы (F5) состояние приложения создаётся заново.



Internet Shop
Learn. Build. Deploy.

1 До перезагрузки страницы

Пользователь аутентифицирован



Состояние во фронтенде (в памяти JavaScript)

```
authState = {
  user: {
    id: 1,
    name: "Иван",
    role: "user"
  },
  accessToken: "eyJhbGciOi..."
}
```



Хранится в памяти
страницы
(Heap / JavaScript state)

Перезагрузка страницы



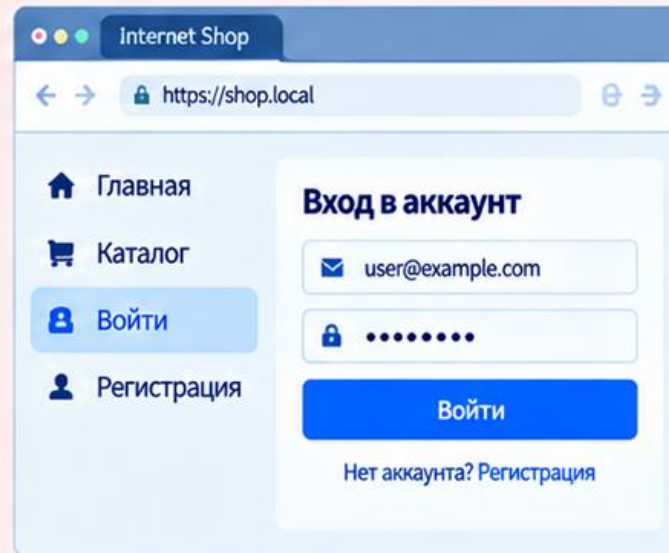
F5

или

Ctrl + R

2 После перезагрузки страницы

Приложение снова в состоянии guest



Состояние во фронтенде (в памяти JavaScript)

```
authState = {
  user: null,
  accessToken: null
}
```



Память страницы
очищена
(новый JavaScript context)

Что работает сейчас

- ✓ Доступ к защищённым страницам (например, /orders)
- ✓ Запросы к API с заголовком Authorization: Bearer <token>
- ✓ Отображение данных пользователя

Что изменилось

- ✗ Токен, хранившийся в памяти, потерян
- ✗ authState создан заново (user = null)
- ✗ Приложение снова в состоянии guest
- ✗ Для доступа к защищённым страницам нужно снова войти в систему

Почему так происходит?

- Все переменные JavaScript хранятся в памяти текущей страницы (browser tab).
- При перезагрузке (F5 / Ctrl + R) текущий JavaScript context уничтожается.
- Создаётся новая страница с чистым состоянием (authState = guest).
- Токен в память не сохраняется между сессиями.

Важно

- Такое поведение — это ожидаемо и безопасно.
- Если нужно сохранять сессию между перезагрузками, используйте другой механизм (например, HttpOnly cookie + /auth/me).
- Не сохраняйте JWT в localStorage — это уязвимо к XSS.

Рекомендация

Для учебного проекта мы храним токен только в памяти страницы. После перезагрузки пользователь входит заново. Для «запоминающейся» сессии рассмотрим cookie-based архитектуру на следующей лекции.

Почему не решаем это localStorage

- localStorage и sessionStorage доступны JavaScript-коду origin.
- При XSS токены из Web Storage можно прочитать.
- OWASP рекомендует не хранить authentication tokens, JWT и session IDs в Web Storage.
- Persisted Authentication требует другой архитектуры, а не простого переноса JWT в localStorage.

Auth bootstrap

- Auth bootstrap — начальная процедура, которая при загрузке приложения определяет, существует ли действительная серверная сессия.
- Frontend сначала ставит status = loading.
- Затем обращается к endpoint текущего пользователя.
- Успех → authenticated.
- 401 → guest.
- Bootstrap особенно естественен при HttpOnly cookie/session архитектуре.

Bootstrap при server-managed cookie

- 1 Application start** `status = loading.`
- 2 GET /api/auth/me** `Browser автоматически отправляет auth cookie.`
- 3 Backend проверяет session** `JavaScript не читает секретную cookie.`
- 4 200 + User** `Frontend устанавливает authenticated.`
- 5 401** `Frontend устанавливает guest.`

Концепция bootstrapAuth()

JAVASCRIPT

```
export async function bootstrapAuth() {
  setLoading();

  try {
    const data = await apiFetch("/auth/me");
    setAuthenticatedUser(data.user);
  } catch (error) {
    if (error.status === 401) {
      setGuest();
      return;
    }

    throw error;
  }
}
```

Что важно

- Код показывает общий принцип.
- Для cookie-based auth accessToken JavaScript не требуется.
- Подробную настройку cookies, CORS и CSRF рассмотрим отдельно.



Учебный вопрос 5. Безопасная browser architecture

Способ хранения Authentication влияет на XSS, CSRF и UX

Три подхода для browser authentication

Подход	Плюс	Ограничение
JWT только в памяти	Token не сохраняется после reload	Пользователь теряет login при reload
JWT в localStorage	Просто переживает reload	Доступен JavaScript и уязвим при XSS
HttpOnly cookie / BFF	Credential недоступен JavaScript	Нужно правильно настроить cookies, CSRF и CORS

Способы хранения токена во frontend-приложении

In-memory token vs localStorage vs HttpOnly cookie/BFF — сравнение и рекомендации



Internet Shop

Learn. Build. Deploy.

	In-memory (JavaScript state)	localStorage / sessionStorage	HttpOnly cookie / BFF
Где хранится	Только в памяти JavaScript приложения (например, authState)	В Web Storage браузера (localStorage или sessionStorage)	В cookie браузера (флаг HttpOnly, Secure, SameSite) или на сервере (BFF — Backend for Frontend)
Сохранение после перезагрузки	Нет Токен исчезает при обновлении страницы (F5) или закрытии вкладки	Да localStorage — сохраняется всегда sessionStorage — только в текущей вкладке	Да Cookie сохраняется до истечения срока действия (или сессии)
Доступность для JavaScript	Да Доступен только внутри текущего JavaScript контекста (в памяти)	Да Любой JavaScript на странице может прочитать токен	Нет Недоступен для JavaScript (флаг HttpOnly)
Риск при XSS (кража токена)	Низкий Токен существует только в памяти. При перезагрузке он исчезает. XSS может использовать токен, пока страница открыта.	Высокий Токен легко прочитать через JavaScript и отправить злоумышленнику.	Низкий Токен недоступен для JavaScript, поэтому не может быть украден через XSS. Но нужно защищаться от CSRF (использовать SameSite, CSRF-токены и др.)
Основное назначение	• Краткоживущие токены (access token) • SPA с обязательным повторным входом после перезагрузки • Учебные проекты, простые приложения	• Сохранение сессии между перезагрузками • Удобство для пользователя • Не рекомендуется для чувствительных токенов (access token)	• Продакшен-приложения • Безопасное хранение токенов • Используется вместе с backend (BFF) или напрямую с API (cookie-based auth)
Плюсы	• Просто реализовать • Нет проблем с XSS-утечкой из хранилища • Подходит для учебных примеров	• Просто использовать • Сохраняется между сессиями • Удобно для пользователя	• Высокая безопасность (от XSS) • Удобно для пользователя • Автоматическая отправка с каждым запросом • Хорошо работает с BFF
Минусы	• Теряется при перезагрузке • Нужно реализовывать повторную аутентификацию/refresh	• Уязвим к XSS • Нельзя хранить refresh token (риск выше) • Нет защиты сам по себе	• Требуется настройка backend (CORS, CSRF) • Сложнее реализовать • Нельзя использовать из чистого frontend без сервера (BFF или cookie-based API)



Рекомендация

Для учебных целей можно использовать in-memory token. Для реальных проектов предпочтителен HttpOnly cookie (часто в связке с BFF). Не используйте localStorage или sessionStorage для хранения чувствительных токенов (access token, refresh token).



Независимо от способа хранения, всегда проверяйте токен на backend. Frontend — это только UX, а не защита!

HttpOnly cookie

- Cookie с HttpOnly недоступна через `document.cookie`.
- Browser может отправлять её автоматически с HTTP-запросами.
- Secure ограничивает отправку HTTPS.
- SameSite управляет cross-site отправкой.
- Cookie снижает риск кражи token через JavaScript, но требует защиты от CSRF в подходящих сценариях.

Fetch и cookies

- Для same-origin requests credentials включаются по умолчанию.
- Для cross-origin cookie requests обычно требуется credentials: include.
- При cross-origin сценарии сервер также должен корректно настроить CORS.
- SameSite может запретить отправку cookie независимо от credentials.
- Подробно разберём это на следующем занятии.

Reauthentication для чувствительных действий

- Даже `authenticated session` не означает, что любая критичная операция должна выполняться без повторной проверки.
- Смена `password` или `email` может требовать повторный ввод `credentials`.
- Платёжные и другие критичные действия могут требовать дополнительное подтверждение.
- Frontend должен уметь показать `reauthentication flow`, но решение принимает backend.

Frontend authentication checklist

- ✓ Иметь единый auth state
- ✓ Использовать состояния loading, guest и authenticated
- ✓ Использовать route guards только для UX
- ✓ Не считать role во frontend доказательством права
- ✓ Централизованно обрабатывать 401
- ✓ Не делать logout при 403
- ✓ Очищать sensitive UI при logout
- ✓ Не хранить JWT в localStorage/sessionStorage
- ✓ Для persisted session проектировать HttpOnly cookie/BFF
- ✓ Всегда оставлять реальную проверку backend

Мини-практика: protected frontend

Добавьте auth state, hash-routing и guards для страниц Internet Shop.

Задание

- Создать состояния guest/loading/authenticated
- Создать routes #/, #/login, #/orders, #/profile, #/admin
- Реализовать requireUser()
- Реализовать requireAdmin()
- Перерисовывать navigation после login/logout
- Обработать 401 как потерю Authentication
- Обработать 403 без logout
- Проверить поведение после reload

Критерии проверки

- Guest не открывает protected pages через UI
- User открывает orders/profile
- User не открывает admin UI
- Admin видит admin navigation
- 401 переводит frontend в guest
- 403 оставляет пользователя authenticated
- Backend endpoints по-прежнему защищены независимо от frontend

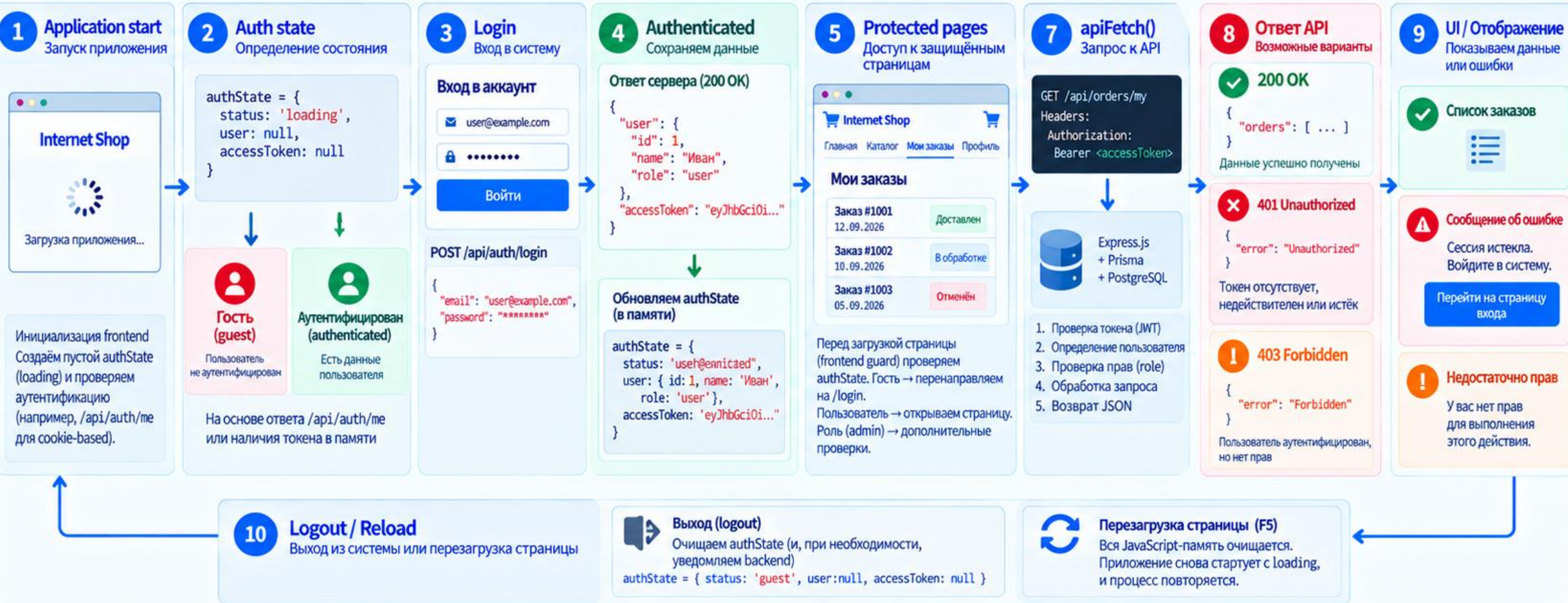
Полный frontend authentication lifecycle — Internet Shop

От запуска приложения до выхода из системы. Основные состояния, запросы и ответы (200 / 401 / 403).



Internet Shop

Learn. Build. Deploy.



Основные состояния authState



Коды ответов и их смысл

- ✓ **200 OK** Запрос выполнен, возвращаются данные
- ✗ **401 Unauthorized** Токен отсутствует, недействителен или истёк
- ! **403 Forbidden** Пользователь аутентифицирован, но нет прав

Важно помнить

- Route guard во frontend улучшает UX, но не заменяет проверку на backend.
- Токен хранится только в памяти (или в cookie при BFF).
- После перезагрузки страницы состояние теряется.
- Все защищённые запросы всегда проверяются на backend.
- Используйте правильные коды ответов: 200 / 401 / 403.

Результат

Безопасный и понятный жизненный цикл аутентификации во frontend-приложении Internet Shop.

Главная граница доверия

- Frontend-код выполняется на устройстве пользователя.
- Пользователь может изменить DOM, JavaScript и запросы.
- Поэтому frontend auth state нельзя использовать как окончательное доказательство доступа.
- Backend проверяет credentials, session/token, role и ownership.
- Frontend отражает результат этих проверок в интерфейсе.

Контрольные вопросы

1. Чем frontend auth state отличается от server-side Authentication?
2. Зачем нужно состояние loading?
3. Что делает route guard?
4. Почему route guard не защищает REST API?
5. Что должен сделать frontend при 401?

6. Почему при 403 не нужно выполнять logout?
7. Что происходит с in-memory JWT после reload?
8. Почему localStorage не является безопасным хранилищем authentication token?
9. Что такое auth bootstrap?
10. Зачем нужны HttpOnly cookies или BFF для persisted browser session?

Итоги занятия

1

Frontend хранит UI-состояние Authentication, но backend остаётся источником истины.

2

Полезно явно моделировать loading, guest и authenticated.

3

Protected pages используют route guards для навигации и UX.

4

Role-aware UI не заменяет server-side Authorization.

5

401 означает потерю действительной Authentication и обычно переводит UI в guest.

6

403 означает недостаточные права, но Authentication сохраняется.

7

Logout должен очищать state и sensitive UI.

8

In-memory token исчезает после reload.

9

JWT не следует переносить в localStorage только ради persistence.

10

Persisted browser session обычно проектируют через HttpOnly cookie/session или BFF.